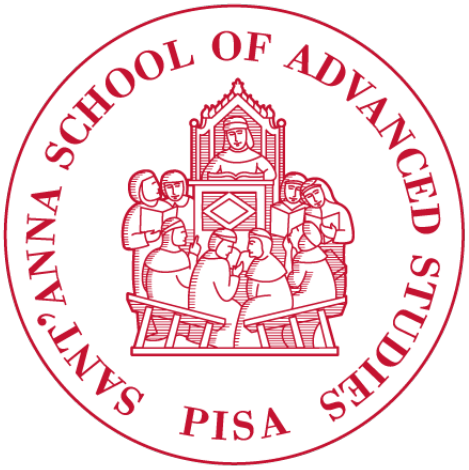

Engineering - Computer and network engineering

Fundamentals of Neural Networks

Self-driving car and obstacle avoidance with Reinforcement Learning

Professor:

Prof. Giorgio Buttazzo

Student:

Francesco Barcherini
f.barcherini@santannapisa.it

ACADEMIC YEAR 2025/2026

May 10, 2026

Contents

1 Introduction

Autonomous driving is a control problem in which a vehicle must identify the drivable region, avoid obstacles, keep a coherent direction of motion and reach a target. This project studies this problem through a 2D simulator and a reinforcement learning controller supported by a Kohonen self-organizing map.

The application consists of a rectangular car moving on user-defined tracks. Each track has a start line, a finish line, road borders and optional circular obstacles. The vehicle observes the environment through lidar-like distance sensors and chooses discrete actions that modify its acceleration and heading. The objective is to reach the finish line while avoiding collisions with both borders and obstacles, and while keeping the trajectory reasonably efficient. The central idea is to combine two learning components. A self-organizing map (SOM), also known as a Kohonen map, discretizes continuous lidar measurements into a finite state index. A tabular Q-learning agent then learns an action value for each discrete state-action pair.

The project was implemented in Python using modular components for track creation, environment simulation, SOM training, reinforcement learning training, inference and quantitative evaluation. The final workflow supports both interactive visual inspection through Pygame and batch evaluation over a collection of test tracks. An example of the system in action is shown in Figure 1: the car (yellow rectangle) is navigating a track with borders (grey lines) and obstacles (red circles); the lidar sensors are represented as blue rays, and the target finish line is shown in yellow.

The report is organized as follows. Section 2 introduces the system architecture and the main specifications. Section 3 describes the simulator, the SOM and the Q-learning algorithm. Section 4 defines the evaluation metrics and presents the generated analysis plots. Finally, Section 5 summarizes the achieved results and discusses possible extensions.

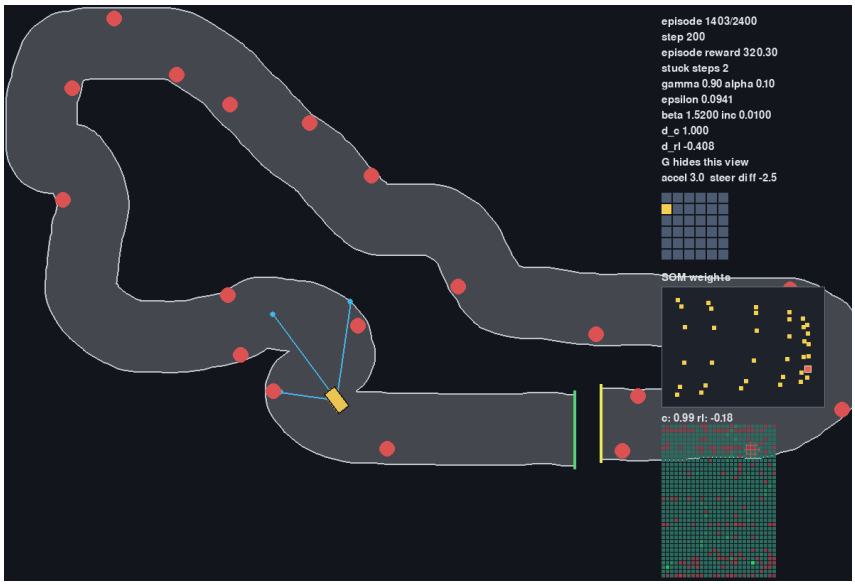

Figure 1: An example of the system in action.

2 System overview and specifications

2.1 Architecture

The system is organized as a closed-loop learning and control pipeline, as shown in Figure 2. The environment provides the current car state and lidar distances. The lidar vector is transformed into a feature vector, normalized and mapped by the SOM to the best matching unit (BMU). This BMU is the discrete state of the Q-table. The policy selects one action from a finite action set, and the simulator applies it to update the vehicle dynamics.

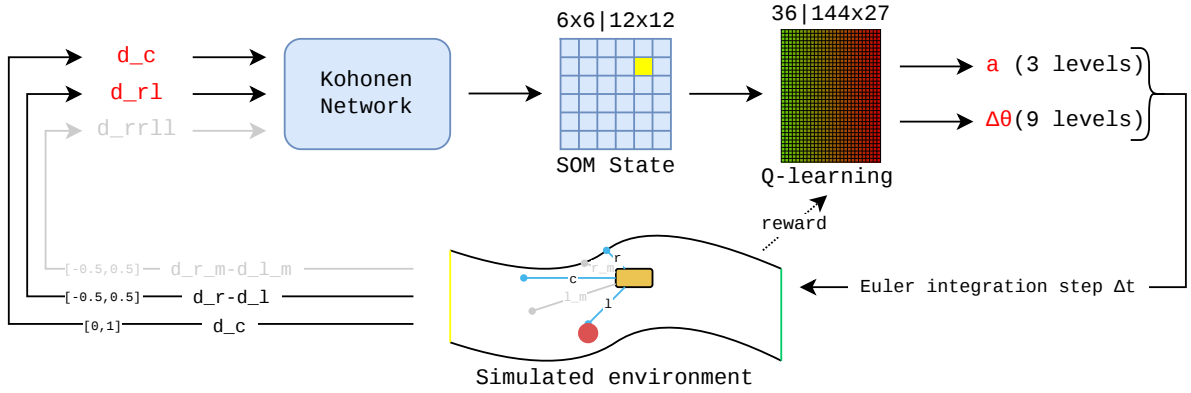

Figure 2: Block diagram of the implemented perception-control loop

The workflow consists of five main phases. The track editor creates and modifies JSON tracks. The SOM training script learns the state discretizer from sampled or policy-generated lidar data. The reinforcement learning script trains and checkpoints Q-tables. The inference script loads trained models and runs the policy with a visual interface. The evaluation scripts compute metrics for trained models and produce plots.

2.2 Environment specification

Each track is a non-loop circuit described by a centerline, a road width, a start line, a finish line and circular obstacles. The drivable road is obtained by buffering the centerline by half the track width.

The car is represented by a rectangular rigid body. Its kinematic state is

$$s_{car} = (x, y, \theta, v),$$

where (x, y) is the center position, θ is the heading angle and v is the scalar longitudinal velocity. The default initial pose is the first centerline position for which the full car rectangle lies inside the road, with heading aligned to the local track direction. This prevents the episode from starting in an invalid off-track state.

The final model uses five lidar rays distributed from $-\alpha$ to $+\alpha$, where α is configured as

45 degrees. The three-lidar version has only left, center and right rays.

2.3 Control specification

The action space contains 27 actions, defined as the Cartesian product of three acceleration values and nine steering-delta values: in this way, the action space takes into account the dependency of a *good* action from the combination of both acceleration and steering. The acceleration values are full braking, no acceleration and full acceleration. The steering component is a direct heading increment rather than an angular acceleration; this choice was introduced because direct steering was more reactive in the discrete controller.

3 Implementation details

3.1 Software structure

The project was implemented in Python. Numerical operations are based on NumPy, geometry and collision checks on Shapely, interactive rendering on Pygame, SOM training on MiniSom, and analysis plots on Matplotlib.

The main entry points cover the complete workflow:

- `draw_tracks.py` creates tracks and edits obstacle layouts;
- `train_som.py` trains the Kohonen map from sampled or policy-generated lidar data;
- `train_rl.py` trains and checkpoints the Q-learning controller;
- `run_inference.py` executes a trained policy;
- `evaluate_models.py` and `plot_analysis.py` compute metrics and generate plots.

Configuration values are collected in `config.toml`; tracks are stored as JSON files, models as NumPy `.npz` checkpoints, and analysis outputs as CSV files and PNG figures.

3.2 Simulation environment

The road geometry is generated from the centerline using a Shapely line buffer. Obstacles are circular polygons. The car rectangle is rotated according to the current heading, and its polygon is used for all collision checks. An episode terminates when the car intersects the finish line, goes off road, hits an obstacle, or reaches the maximum number of steps.

The car dynamics follow the simple explicit Euler integration. At each step,

$$v_{t+1} = min(v_t + a_t \Delta t, v_{\max}), \qquad \theta_{t+1} = \theta_t + \Delta\theta_t,$$

$$x_{t+1} = x_t + v_t \cos(\theta_t)\Delta t, \qquad y_{t+1} = y_t + v_t \sin(\theta_t)\Delta t$$

The lidar system casts rays from the car center and computes the nearest valid intersection with road borders or obstacles. The finish line is treated as a target rather than as an obstacle, so intersections beyond it are ignored.

3.3 Kohonen self-organizing map

The SOM is used to convert continuous lidar measurements into a discrete state with a distribution similar to the input space, as shown in Figure 3. In the three-lidar setting and five-lidar setting, the feature vector is respectively

$$\phi_3 = (d_c, d_r - d_l), \quad \phi_5 = (d_c, d_r - d_l, d_{mr} - d_{ml}),$$

where d_c is the central distance, d_l and d_r are the left and right distances, and d_{ml} and d_{mr} are the middle-left and middle-right distances.

The normalization is component-wise:

$$\hat{d}_c = \frac{\text{clip}(d_c, 0, d_{\max})}{d_{\max}} \in [0, 1], \qquad \widehat{\Delta d} = \frac{\text{clip}(\Delta d, -d_{\max}, d_{\max})}{2d_{\max}} \in [-0.5, 0.5]$$

The main experiments considered 6×6 and 12×12 grids, corresponding to 36 and 144 discrete states. Training samples can be generated randomly from valid track positions, or collected by running an existing Q-table policy and recording simulated lidar measurements: the former approach was used because it better covers outlier scenarios in the state space, such as very close obstacles. The training process adopts a gaussian neighborhood function with inverse time decay of the learning rate ($\alpha_0 = 1$) and neighborhood variance ($\sigma_0 = \frac{\sqrt{N_{neurons}}}{2}$). The final SOM weights are stored as a NumPy array checkpoint.

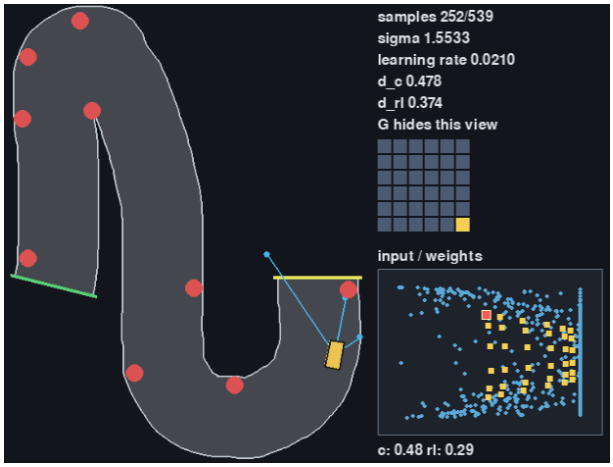

Figure 3: Training of the SOM weights with random samples in the three-lidar setting.

3.4 Q-Learning controller

For each SOM state and each action, the Q-table stores an estimate of the expected total discounted future return. The update rule is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right],$$

with the terminal case obtained by removing the future-value term. The values of α, γ, ϵ, β, δ and the reward parameters are read from the configuration file. The learning

rate α is constant and equal to 0.1; the discount factor γ is constant and equal to 0.9. During the training phase, a ϵ-greedy strategy is used for action selection combined with Boltzmann exploration. The exploration rate ϵ starts at 1.0 and decays by a factor of $\delta = 0.97$ every episode, while the Boltzmann parameter β starts at 0.0 and increases by 0.01 every episode, thus progressively reducing exploration.

Since the success condition consists of reaching the finish line, the reward is very sparse. To address this, two approaches have been adopted: curriculum learning and reward shaping. Regarding the former, training started on simple tracks without obstacles, then moved to more complex tracks without obstacles, followed by easy obstacle layouts, and finally hard obstacle layouts. Checkpoints store the Q-table, action metadata, linked SOM path, lidar configuration, and recent episode rewards. When resuming from a checkpoint, ϵ and β are both set to 0.5.

The reward function for each step is shaped with the following terms (multiple values are relative to different phases of the curriculum learning):

- a small step penalty to encourage shorter trajectories;
- a large positive terminal reward for reaching the finish line;
- a large negative terminal reward for collisions, off-track, or max steps terminations;
- a small positive reward for safe states ($d_c > 0.7$ and $d_{rl} < 0.3$))
- a small negative reward for unsafe states ($d_c < 0.5$ or $d_{rl} > 0.5$);
- a small positive reward for any centerline point crossed in the forward direction;
- a small negative reward for any centerline point crossed in the backward direction;
- a stuck penalty when the car does not progress crossing a centerline point for a configured number (3) of consecutive iterations.

The reward values are indicated in the Table 1 for the different phases of the curriculum learning:

Reward term	Simplest	Non-obs.	Obs. easy/hard
Step penalty	-0.1	-0.1	-0.1
Success reward	100.0	500.0	1000.0
Failure penalty	-100.0	-500.0	-500.0
Safe state reward	0.1	0.1	0.2
Unsafe state penalty	-0.1	-0.1	-0.2
Progress reward	10.0	7.5	10.0
Regress penalty	-10.0	-7.5	-10.0
Stuck penalty	-1.0	-1.0	-1.0

Table 1: Reward parameters for the different phases of curriculum learning

4 Evaluation and results

4.1 Experimental setup

The trained models have been evaluated on the saved test tracks. Each run starts from the deterministic initial pose and applies the greedy action with maximum Q-value. The evaluation records terminal reason, trajectory length, number of steps, lidar safety values and geometric distances from borders and obstacles.

The compared configurations are summarized in Table 2. They represent the main design choices explored during development: SOM distance metric, steering range, number of SOM states and number of lidar rays.

Table 2: Main trained model configurations.

Label	Lidar	SOM	Notes
`som_cosine`	3	6×6	cosine distance, $\pm 2^\circ$
`3-lidar`	3	6×6	Euclidean distance, $\pm 2^\circ$
`3-lidar-5deg`	3	6×6	wider steering, $\pm 5^\circ$
`3-lidar-5deg-12som`	3	12×12	larger state space
`5-lidar-5deg`	5	6×6	5 lidar rays

4.2 Metrics

The following metrics were computed for each model-track pair: training reward, success rate, distance percentage, unsafe value, safety margin, weighted time ratio.

4.2.1 Training reward

Figure 4: Moving average of training reward by episode.

Figure 4 reports the reward evolution saved in the Q-table checkpoints. The reward is averaged over the last window of episodes covering every track. The plot is intended to show both learning stability and the impact of curriculum changes. Cross markers indicate checkpoints where the curriculum dataset changed, which often causes a drop in reward due to the new challenges introduced. The figure reports training values until the episodes with stabilized reward. It is possible to see that the 5-lidar model has a more stable evolution but, having more parameters, it converges after more episodes than the 3-lidar models. For the next metrics, only the final checkpoint trained on hard obstacles is considered for each configuration.

4.2.2 Success rate and distance percentage

On the 45 test tracks, the last of which can be considered quite challenging, the success rate is low for the two simplest models (som_cosine and 3-lidar) and increases at 71.7% for the remaining models. The som with grid dimension of 12 neurons shows slightly better performance at failures compared to the 6 neuron grid.

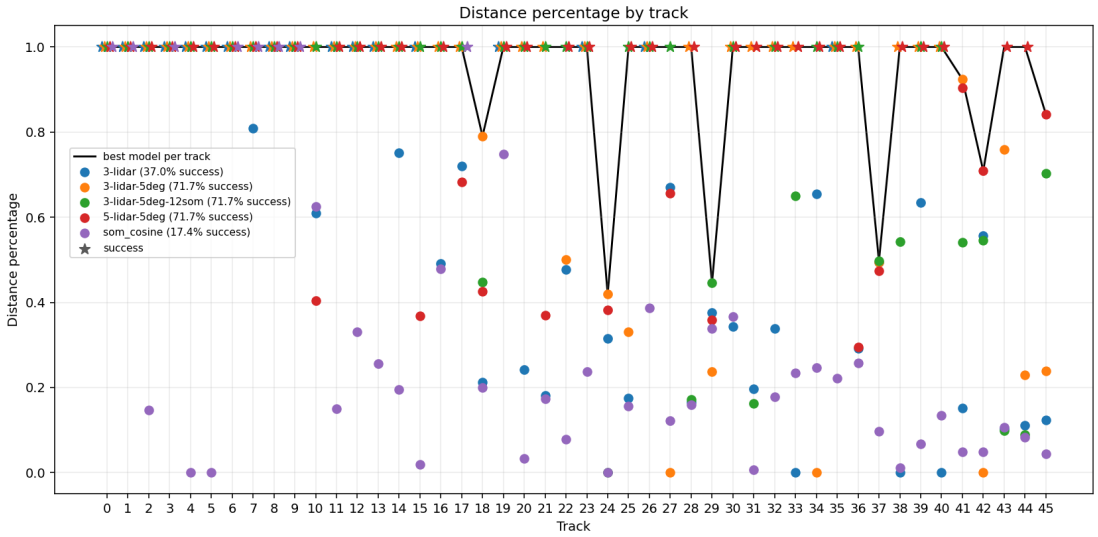

Figure 5: Distance percentage reached by each model on each test track in case of failure.

Figure 5 shows the distance percentage reached by each model on each test track, computed as the ratio of the distance traveled to the total length of the centerline of the track. The 5-lidar model shows a significant improvement in hard tracks while suffering from more early failures in the easier tracks mainly due to specific complex patterns (e.g., sharp turns with close obstacles) that have not been learned during training, probably because of the increased input space and the limited number of training episodes. The cosine similarity model is problematic because ignores the magnitudes of the lidar distances and takes into account only their relative proportions. The model with 2° steering range is not able to react to sharp turns and obstacles. These last models will not be considered for the next metrics.

4.2.3 Unsafe value and safety margin

Unsafe value is a metric computed to measure the average risk of collision following models' trajectories, based on unsafeness thresholds used for reward shaping. It is defined as

$$unsafe\_value = \frac{\sum_{unsafe} max(1 - \hat{d}_c, \frac{|\hat{d}_{rl}|}{0.5})}{N_{steps}}$$

thus computing the average unsafeness of steps; the unsafeness is the maximum between the central proximity to an obstacle and the normalized lateral imbalance.

The safety margin is, instead, the minimum distance between the car rectangle and any obstacle or road border, after ignoring the first 200 steps to avoid measuring the initial distance from the starting line. Unsafe value captures perception-based risk, while safety margin captures geometric clearance.

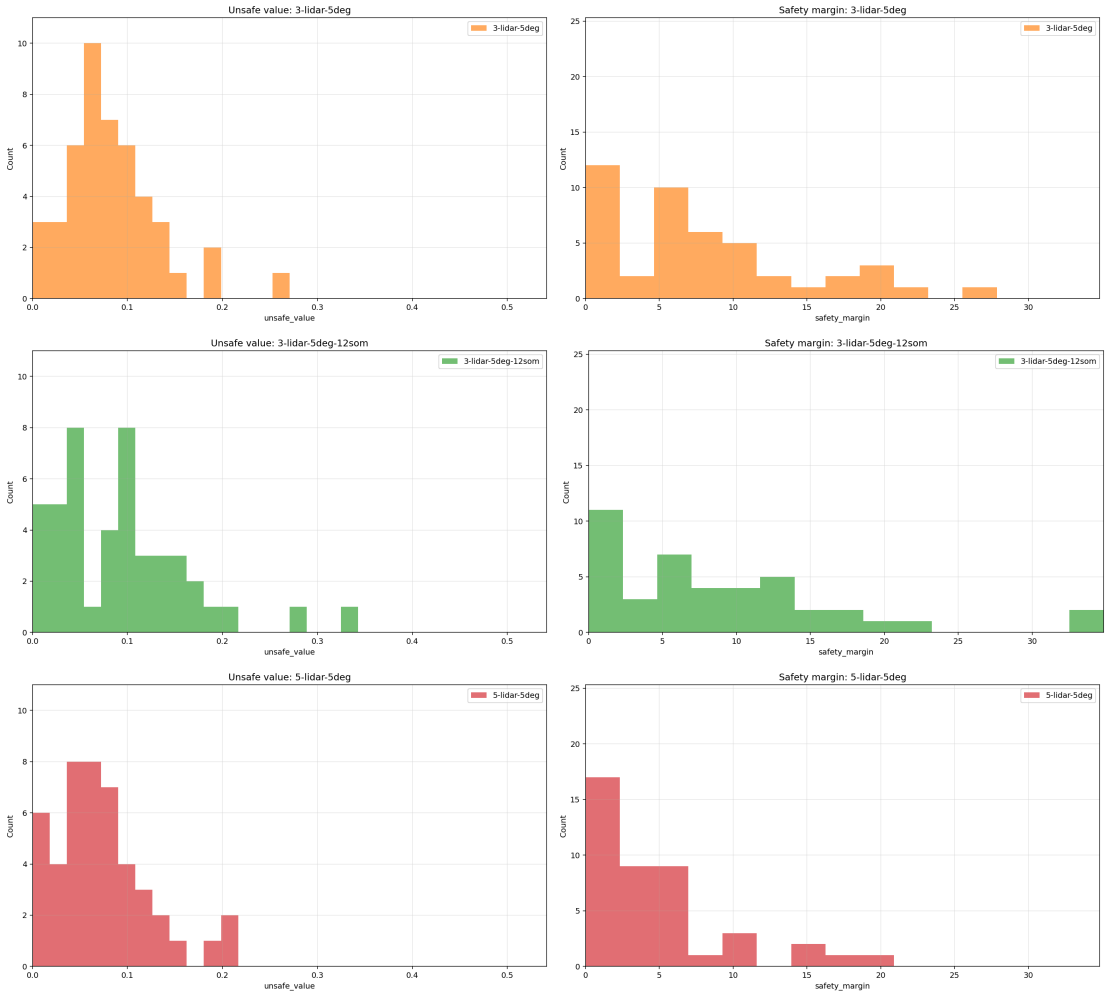

Figure 6: Comparison of the distributions of unsafe value and safety margin across models.

Figure 6 summarizes the distributions of the two metrics across the models. The unsafe value (lower is better) is similar for all models, with a slightly left skewed distribution for the 5-lidar model. On the other hand, the safety margin (higher is better) shows a thinner

margin for the 5-lidar model. Considering that the success rate of the 5-lidar model is high, it is possible to conclude that the model is able to reach the goal while taking more risks and driving closer to obstacles and borders, but for less steps in unsafe states. This is a consequence of the reward shaping, which encourages the model to prioritize progress and speed in reaching the goal along with safety.

4.2.4 Weighted time ratio

In order to compare the efficiency of the models, the weighted time ratio is computed for model m and track i against the base model with 5 lidar rays as

$$time\_ratio_{m,i} = \frac{\sum_i steps_{m,i}/distance\_percentage_{m,i}}{\sum_i steps_{5lidar,i}/distance\_percentage_{5lidar,i}}$$

so that, in case of failure at different completion levels, the number of steps is weighted by the distance percentage itself.

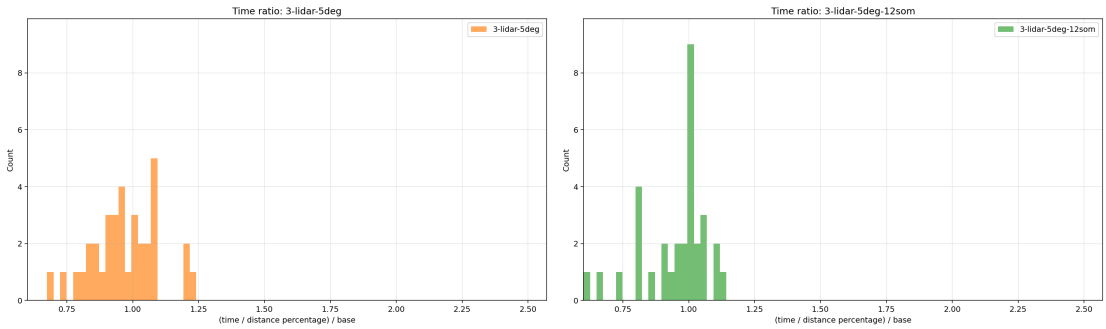

Figure 7: Comparison of the distributions of time ratio against the 5-lidar model.

Figure 7 shows the distribution of the time ratio (left skewed is better) for the 3-lidar-5deg and 3-lidar-5deg-12som models compared to the 5-lidar model (value 1.0). The graph shows that the three models are almost equivalent in terms of efficiency, with the 5-lidar model being slightly faster on average.

5 Conclusions and future work

This project developed a complete two-dimensional self-driving car demonstration based on a Kohonen self-organizing map and tabular Q-learning. The implemented system includes an interactive track editor, a geometric simulator, lidar-based perception, SOM state discretization, reinforcement learning training, live inference and a reproducible evaluation pipeline.

The experiments explored multiple configurations. The most relevant design factors were the number of lidar rays, the steering range and the size of the SOM grid. Richer perception, especially with five lidar rays, provides additional information about obstacles in front of the vehicle and about the symmetry of free space around the car. Larger SOM grids

provide more representational capacity, although they also increase the number of samples and training episodes that must be collected.

Several limitations remain. Central obstacles are particularly difficult because the immediate lidar state may indicate danger without specifying a unique best side for avoidance. The discrete steering policy can also produce oscillations, especially when the car alternates between similar SOM states. Very sharp curves, long tracks and dense obstacle layouts remain challenging for a purely tabular controller.

Future work could extend the project in several directions:

- use SARSA(λ) or Q(λ) to propagate reward information more effectively along trajectories;
- improve reward shaping for bypassing central obstacles;
- explore richer action spaces, including reverse motion and smoother steering commands;
- train larger SOMs or alternative state encoders;
- replace the tabular controller with deep reinforcement learning methods such as DQN or DDPG;
- generate larger benchmark track sets programmatically;
- transfer the approach to a more realistic simulator or to a small robotic platform.

List of Figures

List of Tables